

PromptMeGood — Final Consolidated Replit Handoff

Prepared by: Manus AI

Date: May 2026

Status: Production-ready — all issues confirmed against live DOM and source code

Target: Bring PromptMeGood from 7.1/10 to a polished 10/10 product

This document consolidates every outstanding fix, feature wire-up, and aesthetic improvement into a single, prioritized handoff. Work through the priorities in order. Each section identifies the exact file, element, and code change required.

Summary Table

Priority	Issue	File(s)	Severity
1.1	Buried generate button / collapsed Prompt Tuning	index.html , pmg-fixes-v6.js	Critical
1.2	Visual Studio modal not opening	pmg-visual-studio.js , pmg-visual-studio.css	Critical
2.1	Homepage hero gap (471px empty space)	pmg-fixes-v6.js	High
2.2	Duplicate "Help Me Start" button in nav	index.html	High
2.3	"Advanced Output Settings" expanded by default	index.html	High
2.4	Visual Studio mobile inline layout broken	pmg-visual-studio.css	High
3.1	Storyboard button fires silently (no backend)	ai.ts , pmg-storyboard.js	High
3.2	Sora Video not gated for free users	pmg-visual-studio.js	Medium

3.3	Image save button fails (CORS)	pmg-visual-studio.js	Medium
4.1	Label renames for clarity	index.html , pmg-fixes-v6.js	Low

Priority 1: Critical Blockers

1.1 Buried Generate Button & Collapsed Prompt Tuning

Diagnosis: The "Fix My Prompt" button (`#generateBtn`) is positioned at approximately 5,477px from the top of the page — far below any reasonable fold. The cause is `#settingsPanel` (the Prompt Tuning section), which is a `<section>` element with class `.settings-panel.pmg-prompt-tuning.pmg-stack-card` . This section is **1,904px tall** and sits directly between the goal textarea and the generate button in the DOM. It has no native collapse mechanism — it is always fully expanded. Additionally, `pmg-fixes-v3.js` injects a "Dice Idea Generator" button and extra content into this panel, making it even taller.

The user's experience: they see the goal textarea, then scroll through nearly 2,000px of tone/category/format controls before reaching the generate button. They perceive that the generate button is missing and that the tuning features are an impenetrable wall.

The Fix — Two-Part Approach:

Part A: Add a Mobile Accordion Collapse to `#settingsPanel`

The `#settingsPanel` has a `.pmg-stack-card-head` child div that contains the "Step 1 /  Prompt Tuning" heading. This heading has no click handler. Add a collapse toggle to it.

Add to `pmg-fixes-v6.js` inside `injectStyles()` :

CSS

```
/* ===== FIX: Mobile Prompt Tuning Accordion ===== */
'@media (max-width: 900px) {' ,
'  #settingsPanel.pmg-prompt-tuning { overflow: hidden; transition: max-height 0.3s ease-in-out; }
'  #settingsPanel.pmg-prompt-tuning.pmg-tuning-collapsed { max-height: 72px !important; }
'  #settingsPanel.pmg-prompt-tuning .pmg-stack-card-head { cursor: pointer; user-select: none; }
'  #settingsPanel.pmg-prompt-tuning .pmg-stack-card-head::after { content: "▼"; }
'  #settingsPanel.pmg-prompt-tuning:not(.pmg-tuning-collapsed) .pmg-stack-card-head::after { content: "▲"; }
' },
```

Add a new function to `pmg-fixes-v6.js` and call it from `init()` :

JavaScript

```
function initMobilePromptTuning() {
  if (!once('mobilePromptTuning')) return;
  var tuningPanel = document.getElementById('settingsPanel');
  if (!tuningPanel) return;

  function applyCollapse() {
    if (window.innerWidth <= 900) {
      tuningPanel.classList.add('pmg-tuning-collapsed');
    } else {
      tuningPanel.classList.remove('pmg-tuning-collapsed');
    }
  }

  applyCollapse();

  var header = tuningPanel.querySelector('.pmg-stack-card-head');
  if (header) {
    header.addEventListener('click', function () {
      if (window.innerWidth <= 900) {
        tuningPanel.classList.toggle('pmg-tuning-collapsed');
      }
    });
  }

  window.addEventListener('resize', applyCollapse);
}
```

Part B: Add a Sticky "Fix My Prompt" Button Above the Fold

In addition to the accordion, pin the generate button so it is always visible on mobile regardless of scroll position.

Add to `pmg-fixes-v6.js` inside `injectStyles()` :

CSS

```
/* ===== FIX: Sticky Generate Button on Mobile ===== */
'@media (max-width: 900px) {' ,
'  #generateBtnTop { position: sticky !important; top: 8px !important; z-index: ' ,
'  },
```

Important: The `#generateBtnTop` button is the sticky version in the `.pmgv2-composer-wrap` layout. The primary `#generateBtn` is inside `#tour-step-generate`. Both should be visible and functional. The sticky button ensures the user can always generate without scrolling.

1.2 Visual Studio Modal Not Opening

Diagnosis: Clicking "🎨 Image Studio" (or "Open Visual Studio") fails to open the full-screen modal. The `openVisualStudio()` function in `pmg-visual-studio.js` correctly removes the `hidden` attribute from `#pmg-visual-studio-modal`, but the modal remains invisible due to a CSS specificity conflict. The rule `#pmg-visual-studio-modal[hidden] { display: none !important; }` is not being fully overridden when `hidden` is removed.

Fix 1: Force display in JS — `pmg-visual-studio.js`

Locate the `openVisualStudio()` function and add one line:

JavaScript

```
function openVisualStudio(opts) {
  var m = document.getElementById('pmg-visual-studio-modal');
  if (!m) return;

  m.removeAttribute('hidden');
  m.style.display = 'flex'; // ADD THIS LINE – forces the modal visible
  m.style.removeProperty('display'); // Then let CSS take over (optional, try w

  // ... rest of existing function
}
```

Fix 2: Add CSS specificity override — `pmg-visual-studio.css`

Add this rule to ensure the overlay class always wins when `hidden` is absent:

CSS

```
#pmg-visual-studio-modal.pmg-modal-overlay:not([hidden]) {
  display: flex !important;
}
```

Priority 2: Layout & Aesthetic Polish

2.1 Homepage Hero Gap

Diagnosis: A ~471px empty gap exists between the hero text and the feature cards on the homepage. The `<main class="hero">` element has `flex-grow: 1`, forcing it to stretch and fill all available vertical space in the viewport.

Fix — `pmg-fixes-v6.js` inside `injectStyles()` :

The v6 styles already include a hero fix block. Update the existing rule:

JavaScript

```
/* ===== FIX 1: hero polish ===== */
@media (min-width: 920px) {
  .hero { padding-top: clamp(1.5rem, 3vw, 2.5rem) !important; padding-bottom: 0; }
  // ADD flex-grow: 0 to the existing rule above
}
```

2.2 Duplicate "Help Me Start" Button

Diagnosis: The "Help Me Start" button (`#pmg-help-me-start-btn`) appears both in the top navigation bar and above the goal text area. The `pmg-fixes-v6.js` `reorderHelpMeStart()` function is designed to move the button below `#generateBtn` , but the nav bar instance remains.

Fix — `index.html` :

Locate the `<nav>` element (or the top bar) and find the "Help Me Start" button instance inside it. Remove only that instance. The primary button inside `#prompt-form` (with the "MOST LOVED" badge) must remain.

Alternatively, add a CSS rule to `pmg-fixes-v6.js` to hide the nav bar instance:

CSS

```
/* Hide the duplicate Help Me Start in the nav bar */
.pmgv2-topbar #pmg-help-me-start-btn, .pmgv2-top-actions #pmg-help-me-start-btn { display: none; }
```

2.3 "Advanced Output Settings" Expanded by Default

Diagnosis: The "Advanced Output Settings" section (Money Mode, Human Voice, Clarity Boost) inside `#settingsPanel` is a `<details>` element with class `.advanced-options-summary` . It is currently expanded by default, presenting a wall of text to new users before they have generated their first prompt.

Fix — `index.html` :

Locate the `<details>` element containing the Advanced Output Settings. Ensure it does NOT have the `open` attribute in the HTML. Also, in the `applyExpertMode()` function (around line 5944), the code `advanced.open = true` only fires when Expert Mode is explicitly toggled — this is correct behavior. Do not change it. Simply ensure the initial HTML state is `<details class="advanced-options">` without `open` .

2.4 Visual Studio Mobile Inline Layout

Diagnosis: The mobile inline view of the Visual Studio (when the Visual tab is tapped) is functionally working but aesthetically broken. The "Launch Image Studio" button overlaps the header, and the "Sync with Text Prompt" toggle takes up prime real estate at the top of the screen, pushing the actual image creation tools down.

Fix — `pmg-visual-studio.css` :

CSS

```
/* Hide the redundant launch button when already in inline mode */
@media (max-width: 900px) {
  .pmg-vs-inline #pmg-vs-launch-rail,
  .pmg-vs-inline .pmg-vs-launch-btn {
    display: none !important;
  }
}

/* Move Sync toggle and Master Plan to the bottom */
@media (max-width: 900px) {
  .pmg-vs-inline .pmg-vs-sync-row,
  .pmg-vs-inline .pmg-vs-master-plan {
    order: 99;
    margin-top: 24px;
  }
}

/* Ensure the image/video toggle and goal input are at the top */
.pmg-vs-inline .pmg-vs-mode-bar { order: 1; }
.pmg-vs-inline .pmg-vs-goal-wrap { order: 2; }
.pmg-vs-inline .pmg-vs-photo-suite { order: 3; }
.pmg-vs-inline .pmg-vs-generate-wrap { order: 4; }
}

/* Elevate the Photography Suite header */
.pmg-vs-inline .pmg-photo-suite-header {
  font-size: 1.1rem;
  font-weight: 600;
  color: var(--color-primary, #3ee0a0);
  margin-top: 24px;
  margin-bottom: 12px;
  border-bottom: 1px solid rgba(62, 224, 160, 0.2);
  padding-bottom: 8px;
}

/* Add breathing room between major sections */
.pmg-vs-inline > * + * { margin-top: 16px; }

/* Give the main input more presence */
.pmg-vs-inline textarea {
```

```
min-height: 120px;
font-size: 1.05rem;
line-height: 1.5;
padding: 16px;
}
```

Priority 3: Functional Wiring

3.1 Storyboard Button — Backend Route Missing

Diagnosis: The "🎬 Storyboard" button (`#pmg-generate-storyboard-btn`) fires but produces no visible response. A grep of `api-server/src/routes/ai.ts` and all route files confirms there is **no** `/api/storyboard` **endpoint** in the backend. The frontend call returns a silent 404.

Fix — Backend: `api-server/src/routes/ai.ts`

Add the following route:

TypeScript

```
router.post('/storyboard', requireAuth, async (req, res) => {
  const { goal } = req.body;
  if (!goal) return res.status(400).json({ error: 'Goal is required.' });

  const systemPrompt = `You are a cinematic prompt engineer. Take the user's co

  try {
    const completion = await openai.chat.completions.create({
      model: 'gpt-4o',
      messages: [
        { role: 'system', content: systemPrompt },
        { role: 'user', content: goal }
      ],
      response_format: { type: 'json_object' }
    });
    const data = JSON.parse(completion.choices[0].message.content || '{}');
    res.json({ shots: data.shots || data });
  } catch (e) {
    res.status(500).json({ error: 'Storyboard generation failed.' });
  }
});
```

Fix — Frontend: `pmg-storyboard.js`

Add a loading state and error handling to the storyboard button click handler:

JavaScript

```
document.getElementById('pmg-generate-storyboard-btn').addEventListener('click'
  const btn = this;
  const goal = (document.getElementById('goal') || {}).value || '';
  if (!goal.trim()) {
    alert('Please enter a goal first.');
```

```
    return;
  }

  const originalText = btn.textContent;
  btn.textContent = '⌚ Generating Storyboard...';
  btn.disabled = true;

  try {
    const res = await fetch('/api/storyboard', {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ goal })
    });
    const data = await res.json();
    if (!res.ok) throw new Error(data.error || 'Failed');
    renderStoryboard(data.shots);
  } catch (e) {
    alert('Storyboard generation failed: ' + e.message);
  } finally {
    btn.textContent = originalText;
    btn.disabled = false;
  }
});
```

3.2 Sora Video — Gate for Free Users

Diagnosis: The Video tab in Visual Studio is accessible to all users, but the Sora API is not yet available. Free users clicking "Generate Video" will receive an error.

Fix — `pmg-visual-studio.js` :

At the top of the `generateVideo()` function, add a tier check:

JavaScript

```
async function generateVideo(prompt) {
  // Gate for free users – drive Founding Member urgency
  var isPro = (window.PMG_AUTH && window.PMG_AUTH.tier === 'pro') || false;

  if (!isPro) {
```



```

    // Show upgrade CTA instead of attempting the API call
    showUpgrade('🎬 Video generation is a Founding Member feature. Lock in life
    return;
  }

  // Pro users proceed with the Sora API call
  // ... existing code
}

```

3.3 Image Save Button — CORS Fix

Diagnosis: The "Save" button for generated images sometimes fails because DALL-E 3 URLs expire and fail CORS checks when downloaded directly via an `<a>` tag with a `download` attribute.

Fix — `pmg-visual-studio.js` inside `showImageResult()` :

JavaScript

```

var save = document.getElementById('pmg-vs-save-media');
if (save) {
  save.href = url;
  save.target = '_blank'; // Open in new tab instead of forcing download
  save.removeAttribute('download'); // Remove the download attribute to avoid CORS
}

```

Priority 4: Label Renames

Apply these text changes to reduce cognitive load for first-time users. These can be applied via `pmg-fixes-v6.js` `titleCaseSweepV6()` or directly in `index.html` .

Current Label	New Label	Location
"Visual Asset Engine / Photography Suite"	"Image Prompt Builder"	<code>pmg-visual-studio.js</code> header
"Open Visual Studio"	"Launch Image Studio"	<code>index.html</code> button text
"Master Link — OFF"	"Sync With Text Prompt"	<code>pmg-visual-studio.js</code> toggle label

"Customize Output
(Optional)"

"Advanced Settings"

index.html settings panel label

Do NOT rename: "Help Me Start" — this label appears in 14+ places and is the most-loved feature name per user feedback.

Priority 5: Three Signature Features

These features are partially scaffolded. Wire them up in order of user impact.

5.1 Reverse Engine (Image → Prompt)

Concept: User uploads an image → app reverse-engineers it into a structured text prompt and pre-fills the Photography Suite settings.

Implementation:

1. **UI Hook:** Inside the Visual Studio modal (Image tab), add a button next to the "Describe Your Image" textarea: [📷 Reverse Engineer an Image] .
2. **Flow:** Clicking opens a file picker (.jpg , .png , .webp). On upload, show "Analyzing image DNA..." loading state.
3. **API Call:** Send the image to the existing GPT-4o Vision endpoint (/api/analyze) with this system prompt:

"You are an expert prompt engineer. Analyze this image and return a JSON object with two keys: `prompt` (a highly detailed, plain-language description that would recreate this image in DALL-E 3) and `suite_settings` (an object mapping the image to Photography Suite categories: Style, Camera & Lens, Lighting & Mood, Composition, Color Palette)."

4. **Result:** Populate the image goal textarea with the returned `prompt` . Programmatically select the matching Photography Suite values from `suite_settings` .

5.2 Prompt DNA (Shareable Prompt Cards)

Concept: Every generated image is wrapped in a shareable "DNA Card" displaying the image and the exact prompt that created it.

Implementation:

1. **UI Hook:** In the Visual Studio result view, add [📄 Download DNA Card] next to the existing save button.

2. Canvas Implementation:

- Create a hidden `<canvas>` element (1200×630px for social media).
- Draw the dark theme background.
- Draw the generated image on the top 60% of the canvas.
- Draw the PromptMeGood logo in the corner.
- Draw the prompt text on the bottom 40% using a clean monospace font.
- Trigger download via `canvas.toDataURL('image/png')` .

3. **Social Share:** Add [↗ Share DNA] using the Web Share API (`navigator.share()`) for mobile users.

5.3 Prompt Storyboard (Text → 5 Images → Video)

Concept: One text prompt generates a 5-panel visual storyboard, exportable as a Sora video brief.

Implementation:

1. **UI Hook:** The "🎬 Storyboard" button (`#pmg-generate-storyboard-btn`) already exists. Wire it up per Priority 3.1 above.
2. **Step 1 (Text → 5 Prompts):** Call `/api/storyboard` (add this endpoint per Priority 3.1).
3. **Step 2 (5 Prompts → 5 Images):** Open a "Storyboard Studio" modal. Display the 5 prompts in a vertical list. Fire sequential API calls to DALL-E 3. Render each image next to its prompt as it returns.
4. **Step 3 (Export to Video):** At the bottom of the completed storyboard, add [🎬 Send to Video Studio] . Clicking concatenates the 5 prompts into a narrative sequence and opens the Visual Studio modal (Video tab) with the sequence pre-filled.

Implementation Order

Work through the fixes in this order to maximize visible impact per session:

1. **Fix 1.1** (Prompt Tuning accordion + sticky generate button) — immediately resolves the #1 user complaint
2. **Fix 1.2** (Visual Studio modal) — unblocks the entire image creation workflow
3. **Fix 2.1** (Homepage hero gap) — first impression improvement
4. **Fix 2.2** (Duplicate button) — quick cleanup
5. **Fix 3.1** (Storyboard backend) — enables the signature feature

6. **Fix 2.3 + 2.4** (Settings panel + Visual tab layout) — polish pass
 7. **Fix 3.2 + 3.3** (Video gate + image save) — functional hardening
 8. **Fix 4.1** (Label renames) — final polish
 9. **Features 5.1 → 5.3** (Signature features) — new capability rollout
-

This document supersedes all previous individual handoff documents. All issues have been confirmed against the live DOM at promptmegood.com and the source code in the [DeShaunJ78/Prompt-Me-Good GitHub repository](https://github.com/DeShaunJ78/Prompt-Me-Good).